



Parul University

FACULTY OF ENGINEERING AND TECHNOLOGY
BACHELOR OF TECHNOLOGY

OPERATING SYSTEM

(303105252)

4th SEMESTER

COMPUTER SCIENCE & ENGINEERING DEPARTMENT

Laboratory Manual

CERTIFICATE

This is to clarify that **Mr. Mehul Mer** with Enrollment No: 2403051240022 has completed her laboratory experiments in the **OPERATING SYSTEM** Laboratory (303105252) From the department of **Computer Science & Engineering** during the academic year 2025-26.



Date of Submission:.....

Staff In charge:.....

Head Of Department:.....

Sr.No	EXPERIMENT	DATE OF PERFORMANCE	DATE OF COMPLETION	Page No.	MARKS	Sig
1	Study of Basic commands of Linux	27-11-25	04-12-25	1-3		
2	Study the basics of shell programming.	11-12-25	18-12-25	4		
3	Write a Shell script to print given numbers sum of all digits.	01-01-26	08-01-26	5		
4	Write a shell script to validate the entered date. (eg. Date format is: dd-mm-yyyy).	08-01-26	22-01-26	6		
5	Write a shell script to check entered string is palindrome or not.	22-01-26	2-2-26	7		
6	Write a Shell script to say Goodmorning/Afternoon/Evening as you log in to system.	29-01-26	05-02-26	8		
7 & 11	7) Write a C program to create a child process. 11) Write a program for process creation using C. (Use of gcc compiler.	05-02-26	12-02-26	9 - 10		
8	Finding out biggest number from given three numbers supplied as command line arguments.	12-02-26	26-02-26	11 - 12		
9	Printing the patterns using for loop.	12-02-26	26-02-26	13		
10	Shell script to determine whether given file exist or not.	26-02-26	05-3-26	14		
12	Implementation of FCFS & Round Robin Algorithm.	05-03-26	12-3-26	15 - 20		
13	Implementation of Banker's Algorithm.	05-03-26	12-03-26	21-23		



PRACTICAL NO: 1

Definition: Study of Basic commands of Linux/UNIX.

Command shell: A program that interprets commands is Command shell.

Shell Script: Allows a user to execute commands by typing them manually at a terminal, or automatically in programs called shell scripts.

A shell is not an operating system. It is a way to interface with the operating system and run Commands.

BASH (Bourne Again Shell)

- Bash is a shell written as a free replacement to the standard Bourne Shell (/bin/sh) originally written by Steve Bourne for UNIX systems.
- It has all of the features of the original Bourne Shell, plus additions that make it easier to program with and use from the command line.
- Since it is Free Software, it has been adopted as the default shell on most Linux systems.

1. ls : lists all files

```
Loading...  
Welcome to Fedora 33 (riscv64)  
  
[root@localhost ~]# ls  
bench.py  hello.c  
[root@localhost ~]#
```

2. ls -l : lists files with its details.

```
[root@localhost ~]# ls -l  
total 8  
-rw-r--r-- 1 root root 114 Dec 26 2020 bench.py  
-rw-r--r-- 1 root root 185 Sep 9 2018 hello.c  
[root@localhost ~]#
```

3. `ls -a` : lists hidden files.

```
[root@localhost ~]# ls -a
.      .bash_logout  .bashrc      .cshrc      hello.c
.      .bash_profile bench.py     .fldev_cfg  .tcshrc
[root@localhost ~]#
```

4. `pwd` : prints working directory

```
[root@localhost ~]# pwd
/root
[root@localhost ~]#
```

5. `clear` : clears whole screen.

```
[root@localhost ~]#
```

6. `date` : prints system's current date and time.

```
[root@localhost ~]# date
Thu Dec 11 02:55:13 PM UTC 2025
[root@localhost ~]#
```

7. `cal` : prints current month's calendar and highlight today's date

```
[root@localhost ~]# cal
    December 2025
Su Mo Tu We Th Fr Sa
    1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30 31

[root@localhost ~]#
```

8. `touch` : creates new files.

```
[root@localhost ~]# touch new.c
[root@localhost ~]# ls
bench.py  hello.c  new.c  new.py
[root@localhost ~]#
```



9. echo : prints content on the screen.

```
[root@localhost ~]# echo "Hello World"
Hello World
[root@localhost ~]#
```

10. cat: prints file content on the screen.

```
[root@localhost ~]# echo "My name is Harsh" > new.py
[root@localhost ~]# cat new.py
My name is Harsh
[root@localhost ~]#
```

11. mkdir : makes new directory.

```
[root@localhost ~]# mkdir JavaProject
[root@localhost ~]# ls
bench.py  hello.c  JavaProject
[root@localhost ~]#
```

12. rmdir : removes directory.

```
[root@localhost ~]# rmdir JavaProject
[root@localhost ~]# ls
bench.py  hello.c  new.c  new.py
[root@localhost ~]#
```

13. mv : changes file name

```
[root@localhost ~]# mv new.c Main.c
[root@localhost ~]# ls
bench.py  hello.c  Main.c  new.py
[root@localhost ~]#
```



PRACTICAL NO: 2

Aim: Study the basics of shell programming.

Code :

```
#!/bin/bash
echo "Enter a first number: "
read n
echo "Enter second number"
read m
mul=$(( n * m ))
echo "Multiplication of $n and $m is : $mul"
```

Output :

```
5
6

Output:

Enter a number:
Enter second number
Multiplication of 5 and 6 is : 30
```

PRACTICAL NO: 3

Aim : Write a Shell script to print given numbers sum of all digits.

Code :

```
#!/bin/bash

echo "Enter a number: "

read n

temp=$n

sd=0

sum=0

while [ $n -gt 0 ]
do

    sd=$(( n % 10 ))

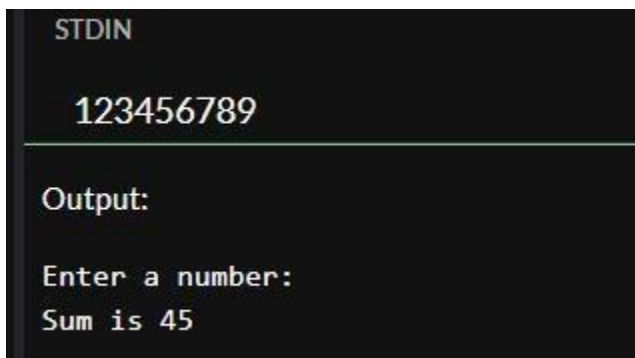
    n=$(( n / 10 ))

    sum=$(( $sum + $sd ))

done

echo "Sum is $sum"
```

Output :



```
STDIN

123456789

Output:

Enter a number:
Sum is 45
```


PRACTICAL NO: 4

Aim: Write a shell script to validate the entered date. (eg. Date format is:dd-mm-yyyy).

Code:

```
#!/bin/bash

read -p "Enter a date(dd-mm-yyyy): " date

if [[ $date =~ ^(0[1-9]|[12][0-9]|3[01])-(0[1-9]|1[0-2])-[0-9]{4}$ ]]; then
    echo "Valid Date"
else
    echo "Invalid Date"
fi
```

Output:



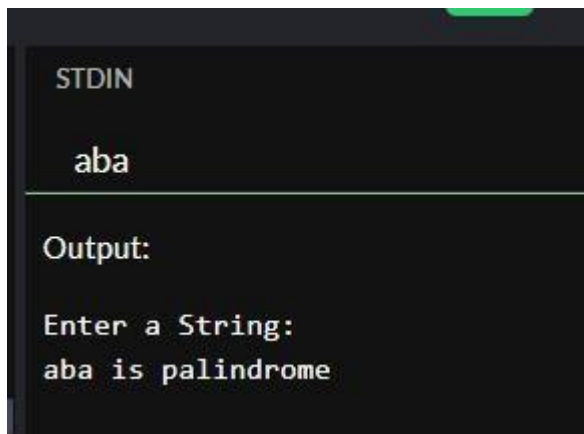
PRACTICAL NO: 5

Aim : Write a shell script to check entered string is palindrome or not

Code :

```
#!/bin/bash  
echo "Enter a String: "  
read name  
name1=$(echo "$name" | rev)  
if [ "$name" == "$name1" ]; then  
    echo "$name is palindrome"  
else  
    echo "$name is not palindrome"  
fi
```

Output :



```
STDIN  
aba  
Output:  
Enter a String:  
aba is palindrome
```



PRACTICAL NO: 6

Aim: Write a Shell script to say Goodmorning/Afternoon/Evening as you log in to system.

Code:

```
#!/bin/bash

hour=$(date +%H)
min=$(date +%M)
ampm=$(date +%P)
echo "$hour:$min $ampm"

if [ "$hour" -lt 12 ]; then
    echo "Good Morning"
elif [ "$hour" -le 16 ]; then
    echo "Good Afternoon"
elif [ "$hour" -le 20 ]; then
    echo "Good Evening"
else
    echo "Good Night"
fi
```

Output:

```
STDIN
Input for the program ( Optional)

Output:
09:26 am
Good Morning
```



PRACTICAL NO: 7 & 11

Aim: Write a c program to create a child process.

Write a program creation using C.

Code:

```
#include <stdio.h>

#include <sys/wait.h>

int main(void)
{
    int pid;
    int status;
    printf("Hello World!\n");
    pid = fork( );
    if(pid == -1) {
        perror("bad fork");
        exit(1);
    }
    if (pid == 0) {
        printf("I am the child process.\n");
    }
    else {
        wait(&status);
        printf("I am the parent process.\n");
    }
}
```



Parul[®]
University

NAAC A++

Output:

PARUL UNIVERSITY
FACULTY OF ENGINEERING & TECHNOLOGY
Operating System (Subject Code) B. Tech. 2nd Year

Output:

Hello World!

I am the child process.

I am the parent process.



PRACTICAL NO: 8

Aim : Finding out biggest number from given three number supply as command line arguments.

Code :

```
#!/bin/bash

echo "Enter first number: "

read n1

echo "$n1"

echo "Enter second number: "

read n2

echo "$n2"

echo "Enter third number: "

read n3

echo "$n3"


if [ "$n1" -gt "$n2" ] && [ "$n1" -gt "$n3" ]; then
    echo "$n1 is biggest number"
elif [ "$n2" -gt "$n1" ] && [ "$n2" -gt "$n3" ]; then
    echo "$n2 is biggest number"
else
    echo "$n3 is biggest number"
Fi
```

Output :

```
STDIN
5
6
9

Output:
Enter first number:
5
Enter second number:
6
Enter third number:
9
9 is biggest number
```

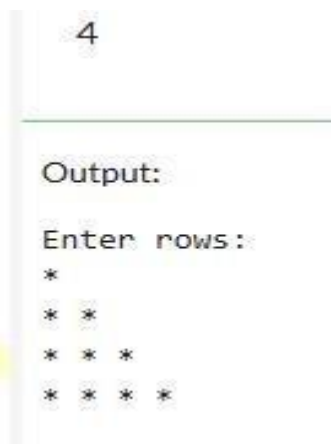
PRACTICAL NO: 9

Aim: Printing the pattern using the for loop.

Code:

```
#!/bin/bash  
echo "Enter rows: "  
read rows  
for ((i=1; i<=rows; i++))  
do  
    for ((j=1; j<=i; j++))  
    do  
        echo -n "* "  
    done  
    echo ""  
done
```

Output:



```
4  
-----  
Output:  
Enter rows: 4  
*  
* *  
* * *  
* * * *
```

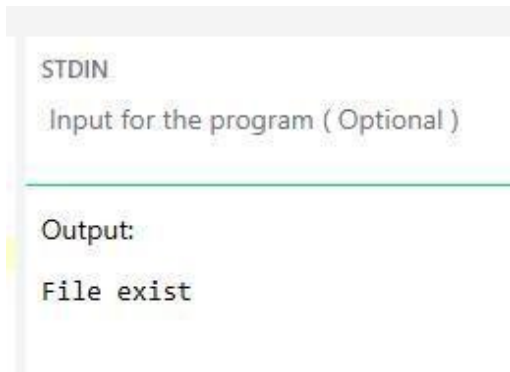

PRACTICAL NO: 10

Aim: Shell script to determine whether given file exist or not.

Code:

```
#!/bin/bash  
if [ -f "script.sh" ];  
then  
    echo "File exist"  
else  
    echo "File doesn't exist"  
fi
```

Output:



```
STDIN  
Input for the program ( Optional )  
  
Output:  
File exist
```

PRACTICAL NO: 12.1

Aim: FCFS algorithm

Code:

```
#include<stdio.h>
```

```
int main(){
```

```
    int bt[10]={0}, at[10]={0}, tat[10]={0}, wt[10]={0}, ct[10]={0};
```

```
    int n,sum=0;
```

```
    float totalTAT=0,totalWT=0;
```

```
    printf("Enter number of processes");
```

```
    scanf("%d",&n);
```

```
    printf("Enter arrival time and burst time for each process\n\n");
```

```
    for(int i = 0; i < n; i++) {
```

```
        printf("Arrival time of process[%d]  ",i+1);
```

```
        scanf("%d",&at[i]);
```

```
        printf("Burst time of process[%d]  ",i+1);
```

```
        scanf("%d",&bt[i]);
```

```
        printf("\n");
```

```
    }
```

```
    for(int j = 0; j < n; j++)
```

```
        { sum+=bt[j];
```



Parul[®]
University

NAAC A++

PARUL UNIVERSITY
FACULTY OF ENGINEERING & TECHNOLOGY
Operating System (Subject Code) B. Tech. 2nd Year

```
ct[j]+=sum;
```

```
}
```

```
for(int k = 0; k < n; k++)
```

```
{ tat[k]=ct[k]-at[k];
```

```
totalTAT+=tat[k];
```

```
}
```

```
for(int k = 0; k < n; k++)
```

```
{ wt[k]=tat[k]-bt[k];
```

```
totalWT+=wt[k];
```

```
}
```

```
printf("Solution: \n\n");
```

```
printf("P#\t AT\t BT\t CT\t TAT\t WT\t\n\n");
```

```
for(int i = 0; i < n; i++) {
```

```
printf("P%d\t %d\t %d\t %d\t %d\t %d\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);
```

```
}
```

```
printf("\n\nAverage Turnaround Time = %f\n",totalTAT/n);
```

```
printf("Average WT = %f\n\n",totalWT/n);
```

```
return 0;
```

```
}
```

Output:

```
Enter number of processesEnter arrival time and burst time for each process
```

```
Arrival time of process[1]    Burst time of process[1]
Arrival time of process[2]    Burst time of process[2]
Arrival time of process[3]    Burst time of process[3]
Arrival time of process[4]    Burst time of process[4]
Arrival time of process[5]    Burst time of process[5]
Arrival time of process[6]    Burst time of process[6]
Arrival time of process[7]    Burst time of process[7]
```

```
Solution:
```

P#	AT	BT	CT	TAT	WT
P1	0	21	21	21	0
P2	0	47	68	68	21
P3	0	35	103	103	68
P4	0	6	109	109	103
P5	0	40	149	149	109
P6	0	2	151	151	149
P7	0	42	193	193	151

```
Average Turnaround Time = 113.428574
```

```
Average WT = 85.857140
```

PRACTICAL NO: 12.2

Aim: Round Robin algorithm

Code:

```
#include<stdio.h>

int main() {

    int bt[10]={0}, at[10]={0}, tat[10]={0}, wt[10]={0}, ct[10]={0};

    int n,sum=0;

    float totalTAT=0,totalWT=0;


    printf("Enter number of processes");

    scanf("%d",&n);


    printf("Enter arrival time and burst time for each process\n\n");


    for(int i = 0; i < n; i++) {

        printf("Arrival time of process[%d]  ",i+1);

        scanf("%d",&at[i]);


        printf("Burst time of process[%d]  ",i+1);

        scanf("%d",&bt[i]);


        printf("\n");

    }

    for(int j = 0; j < n; j++)

        { sum+=bt[j];

          ct[j]+=sum;

        }
```

```
for(int k = 0; k < n; k++)  
    { tat[k]=ct[k]-at[k];  
      totalTAT+=tat[k];  
    }  
  
for(int k = 0; k < n; k++)  
    { wt[k]=tat[k]-bt[k];  
      totalWT+=wt[k];  
    }  
  
printf("Solution: \n\n");  
printf("P#\t AT\t BT\t CT\t TAT\t WT\n\n");  
  
for(int i = 0; i < n; i++) {  
    printf("P%d\t %d\t %d\t %d\t %d\t %d\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);  
}  
  
printf("\n\nAverage Turnaround Time = %f\n",totalTAT/n);  
printf("Average WT = %f\n\n",totalWT/n);  
  
return 0;  
}
```

Output:

Enter number of processesEnter arrival time and burst time for each process

Arrival time of process[1] Burst time of process[1]
Arrival time of process[2] Burst time of process[2]
Arrival time of process[3] Burst time of process[3]
Arrival time of process[4] Burst time of process[4]
Arrival time of process[5] Burst time of process[5]
Arrival time of process[6] Burst time of process[6]
Arrival time of process[7] Burst time of process[7]

Solution:

P#	AT	BT	CT	TAT	WT
P1	0	21	21	21	0
P2	0	47	68	68	21
P3	0	35	103	103	68
P4	0	6	109	109	103
P5	0	40	149	149	109
P6	0	2	151	151	149
P7	0	42	193	193	151

Average Turnaround Time = 113.428574

Average WT = 85.857140

PRACTICAL 13

Aim: Implementation of Banker's Algorithm.

Code:

```
#include<stdio.h>

int main()
{
    int n, m; // n = processes, m = resources

    printf("&quot;Enter number of processes: &quot;);
    scanf("&quot;%d&quot;, &amp;n);

    printf("&quot;Enter number of resources: &quot;);
    scanf("&quot;%d&quot;, &amp;m);

    int allocation[100][100], max[100][100];
    int need[100][100];
    int available[100];
    int finish[100] = {0};
    int safeSequence[100];

    printf("&quot;\nEnter Allocation Matrix:\n&quot;);
    for(int i = 0; i &lt; n; i++)
        for(int j = 0; j &lt; m; j++)
            scanf("&quot;%d&quot;, &amp;allocation[i][j]);

    printf("&quot;\nEnter Max Matrix:\n&quot;);
    for(int i = 0; i &lt; n; i++)
        for(int j = 0; j &lt; m; j++)
            scanf("&quot;%d&quot;, &amp;max[i][j]);

    printf("&quot;\nEnter Available Resources:\n&quot;);
    for(int j = 0; j &lt; m; j++)
        scanf("&quot;%d&quot;, &amp;available[j]);
```



```
// Calculate Need Matrix
for(int i = 0; i < n; i++)
for(int j = 0; j < m; j++)
need[i][j] = max[i][j] - allocation[i][j];
int count = 0;
while(count < n)
{
int found = 0;
for(int i = 0; i < n; i++)
{
if(finish[i] == 0)
{
int possible = 1;
for(int j = 0; j < m; j++)
{
if(need[i][j] > available[j])
{
possible = 0;
break;
}
}
if(possible)
{
for(int j = 0; j < m; j++)
available[j] += allocation[i][j];
safeSequence[count] = i;
finish[i] = 1;
count++;
}
```

```
found = 1;

}

}

}

if(found == 0)

{

printf("&quot;\nSystem is NOT in safe state.\n&quot;);

return 0;

}

}

printf("&quot;\nSystem is in SAFE state.\nSafe Sequence: &quot;);

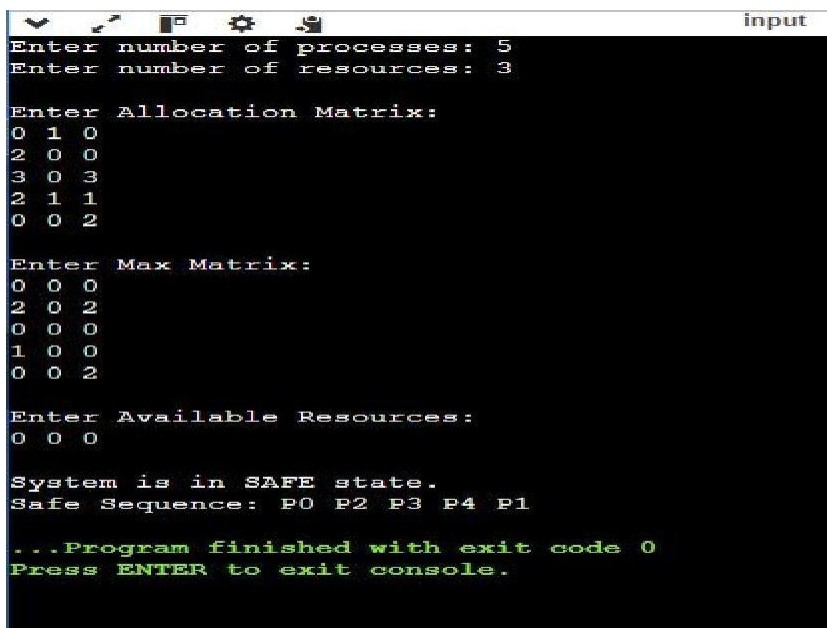
for(int i = 0; i &lt; n; i++)

printf("&quot;P%d &quot;; safeSequence[i]);

return 0;

}
```

Output:



```
input
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Max Matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

Enter Available Resources:
0 0 0

System is in SAFE state.
Safe Sequence: P0 P2 P3 P4 P1

...Program finished with exit code 0
Press ENTER to exit console.
```